700105 Simulation & Concurrency

UNIVERSITY of HULL

Assessment

- Series of lab small exercises throughout
  - You will write up simulation labs in a GitHub classroom
  - You will demonstrate these during the lab sessions
- Single large lab task bringing everything together
  - This is where the majority of the marks come from

What to put in the lab books

- Join GitHub Classroom
  - Links available soon
- Relevant code
  - You can also use this repository as your actual code repository
- Results of tests where appropriate
- Demonstrate that you understand what the code does
  - Description of the techniques used in the code
  - Include diagrams where appropriate

Story So Far

Indicative Content

- Updating State
  - Calculating forces
    - Gravity, Drag, Friction
  - Calculating new velocities and positions
    - Integration methods (Euler, Semi-Implicit Euler, Verlet, Runge-Kutta)
- Collisions - Geometric Objects (Shapes)
  - Primitives (point, line, line segment, plane, sphere, cylinder, capsule, cuboid, triangle, convex hull)
  - Methods (Inside-Outside, Intersects, Contains, SAT, GJK)
- Collision Response
  - Calculating new velocities or forces after collision
  - Conservation of momentum
- Reducing Calculations
  - Spatial Data Structures (uniform grid, quadtree, octree, kd-tree, BSP tree)

Intersection

- We started to look at how we can tell if two shapes have collided
- In this session we'll start to look at how we can move points through space

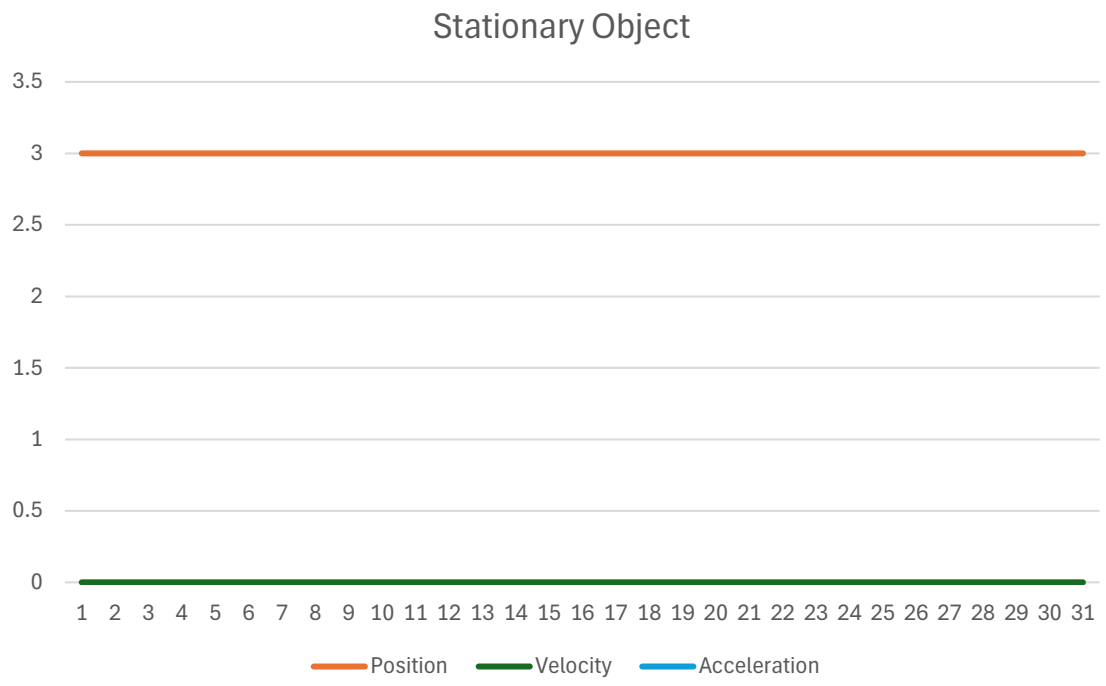
~~Integration~~ Differentiation

Differentiation

- Differentiation tells use how one variable changes with another variable
  - The rate of change of x with respect to y
- Here is the change in position (the y axis) against time (the x axis) of an object moving at 2 units per second
- The gradient of the graph (dx/dt) is 2
  - The rate of change of the position with respect to the time

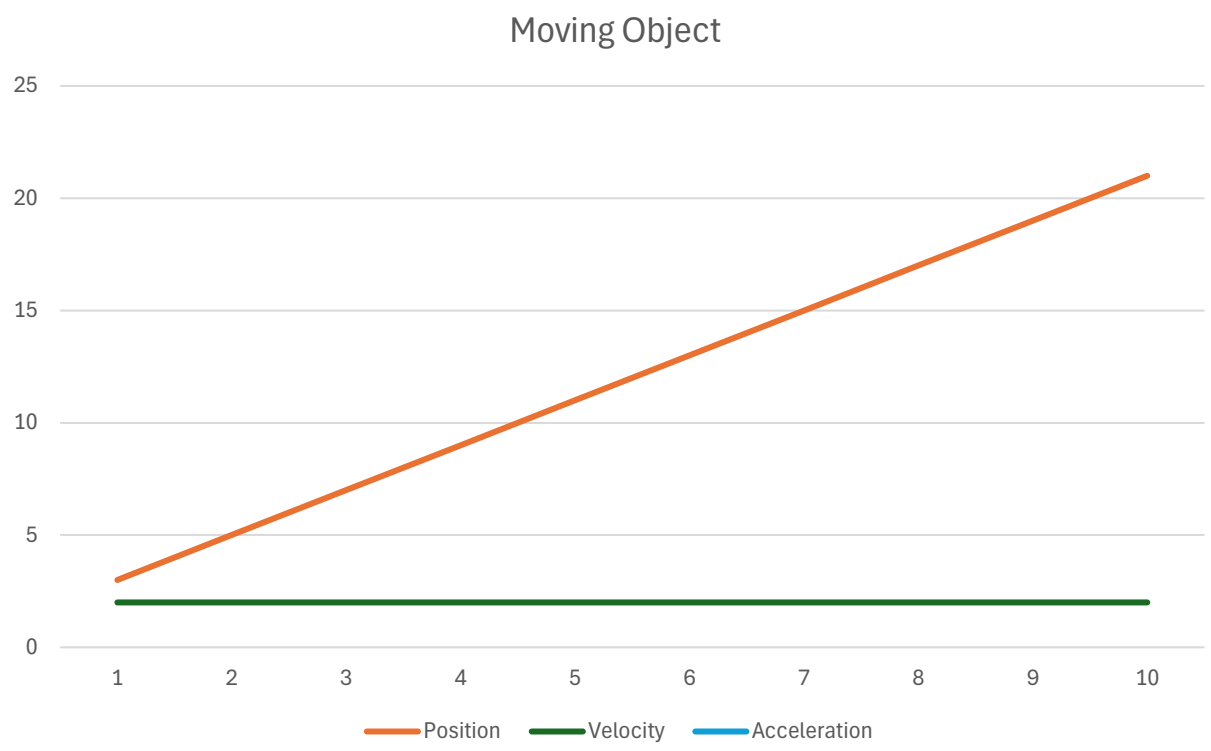
Stationary object

- x(t) = displacement (position) as a function of time
  - units are metres (for example) (m)
- v(t) = velocity (rate of change of position) as a function of time
  - units are metres per second (ms^{-1})
- a(t) = acceleration (rate of change of velocity) as a function of time
  - units are meters per second squared (ms^{-2})



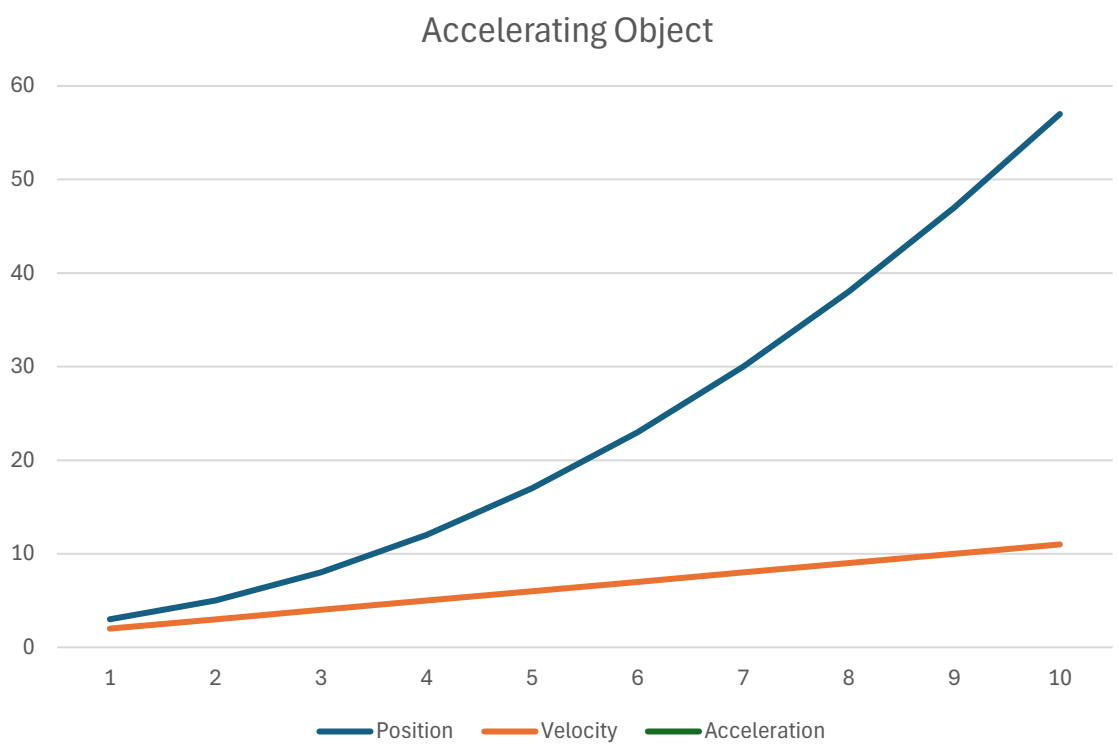
Moving object

- Velocity is constant
- Displacement is a straight line
- $v(t) = dx(t)/dt$
- A large change in position means a large velocity
- No change in position means no velocity



Accelerating object

- Acceleration is constant
  - Often we *assume* this over each timestep
- Velocity is the straight line
  - $a(t) = dv(t)/dt$
  - If velocity is changing quickly (i.e. the line is steep) acceleration is large
  - If the velocity is constant a is zero
- Displacement is the parabolic curve
  - $a(t) = d^2x(t)/dt^2$
  - Second derivative of position with respect to time



Integration

Integration

- Integration is the opposite of differentiation
  - It tells us how much change has built up over time
- It also calculates the area under a graph
- How?
  - By summing up all the little rectangles under the graph and making them very small
- Why?
  - Because often the area under a graph can tell us something interesting about a problem
  - I find looking at the units can give some insight
  - $v * dt = dx/dt * dt = dx$
  - $a * dt = dv/dt * dt = dv$

Acceleration -> Velocity

- How much change in velocity has there been?
- $v(t) = \int a(t)dt + C$
- C is a constant – the velocity at the start

Velocity -> Position

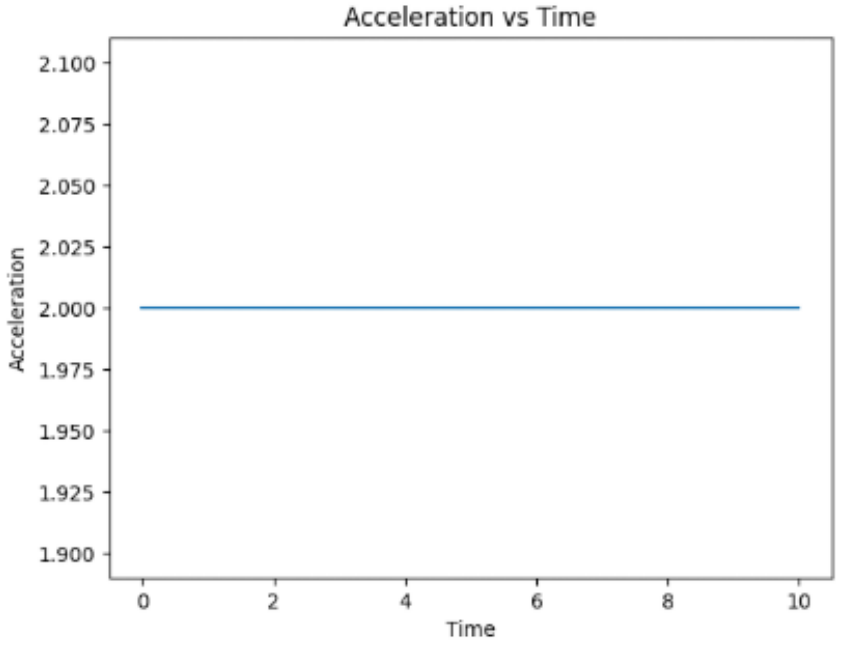
- How much change in position has there been?
- $x(t) = \int v(t)dt + C$
- C is a constant – the displacement (position) at the start

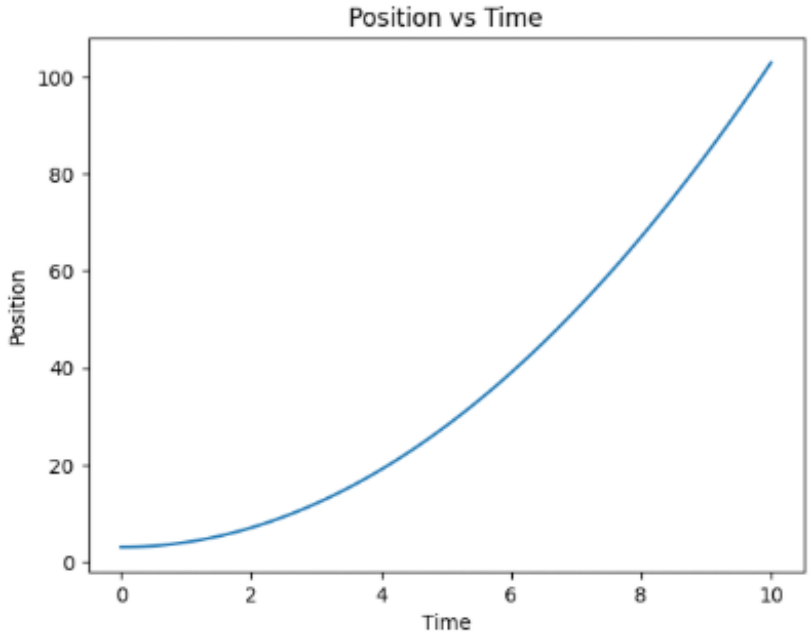
Summary

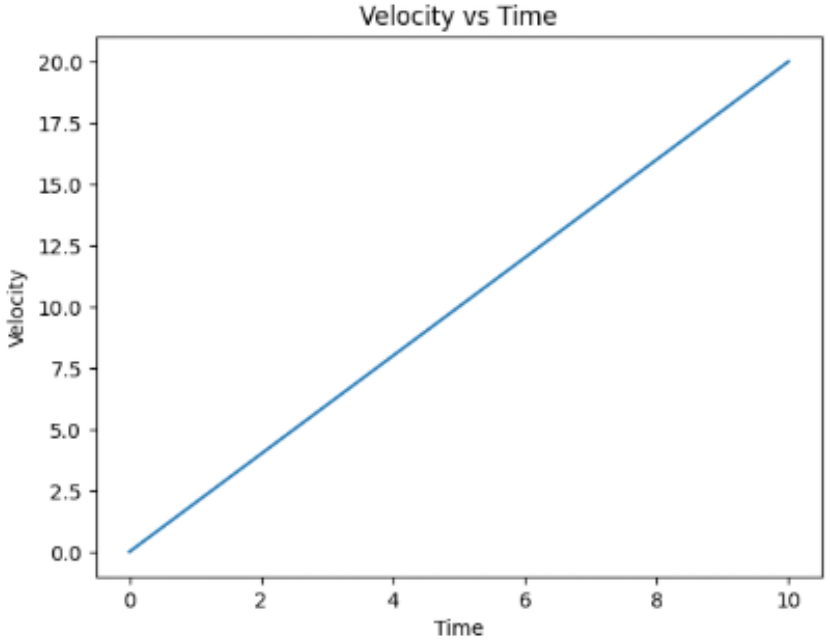
- Differentiation – the slope of the line on a graph
- $x(t) \xrightarrow{\mathrm{d/dt}} v(t) \xrightarrow{\mathrm{d/dt}} a(t)$
- Integration – the area under a line on a graph
- $a(t) \xrightarrow{\int} v(t) \xrightarrow{\int} x(t)$

Examples

- Plot acceleration (ms^{-2}) against time (s)
  - Integral is (ms^{-2} x s) = (ms^{-1}) (velocity)
- Plot velocity (ms^{-1}) against time (s)
  - Integral is (ms^{-1} x s) = (m) (distance/position)







Methods of Integration

Euler Integration

- Assumes that velocity (for example) is constant over delta time
- Calculate the new position based on current velocity
- Calculate new velocity using $v_{n+1} = v_n + a * dt$
- Fast but only works if delta time is very small
- Can result in an increase in energy (which is bad)

Semi-Implicit Euler

- Assumes that velocity (for example) is constant over delta time
- Calculate the new velocity based on current velocity
- Calculate new position based on new velocity
- Fast – and slightly more accurate
  - Still only works if delta time is very small
  - Still can result in an increase in energy (which is bad)

Runge-Kutta

A weighted average of increments

h is the timestep

y_0 is the current position (at time t_0)

y_1 is the next position (at time t_1)

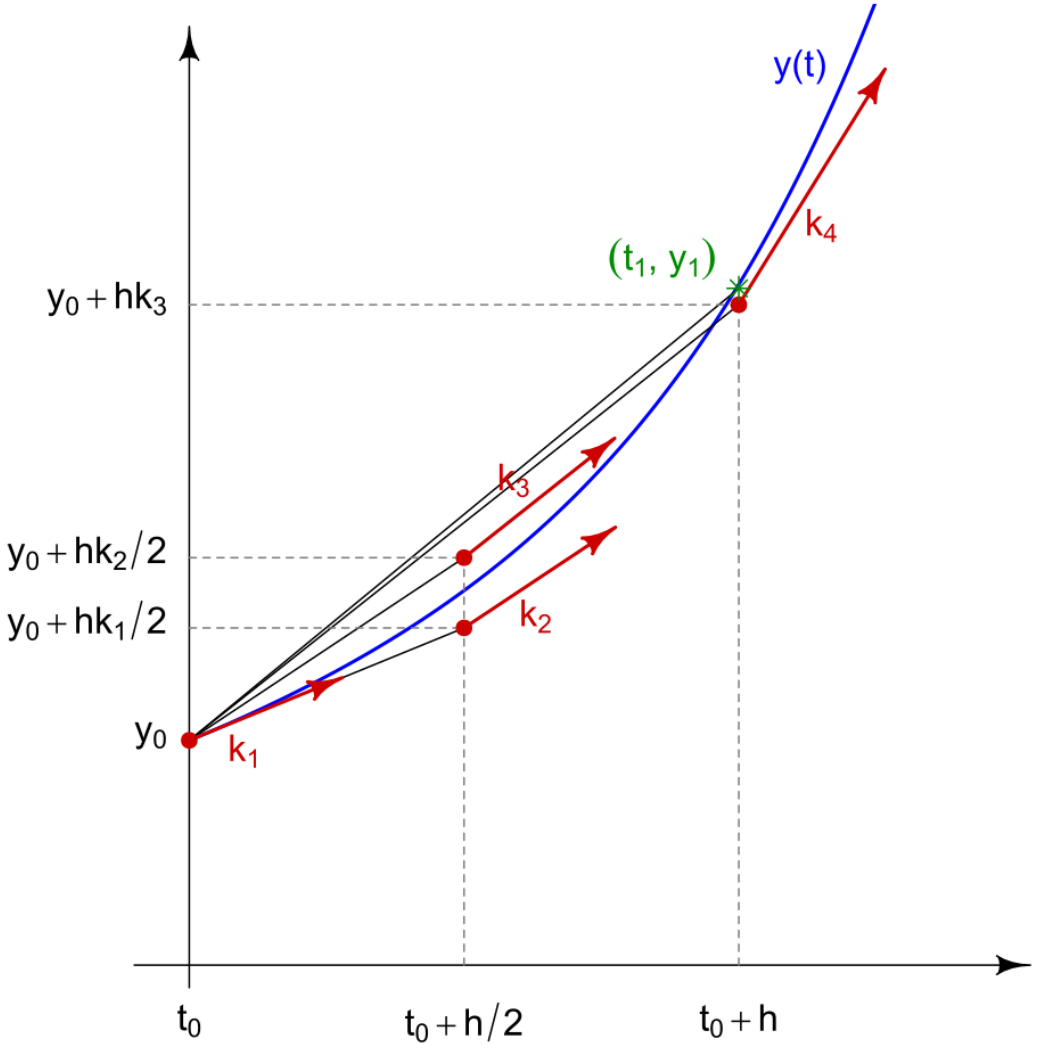
$y_1 = y_0 + (h/6) * (k_1 + 2k_2 + 2k_3 + k_4)$

$k_1 = v_0$

$k_2 = v_0 + (h/2) * k_1$

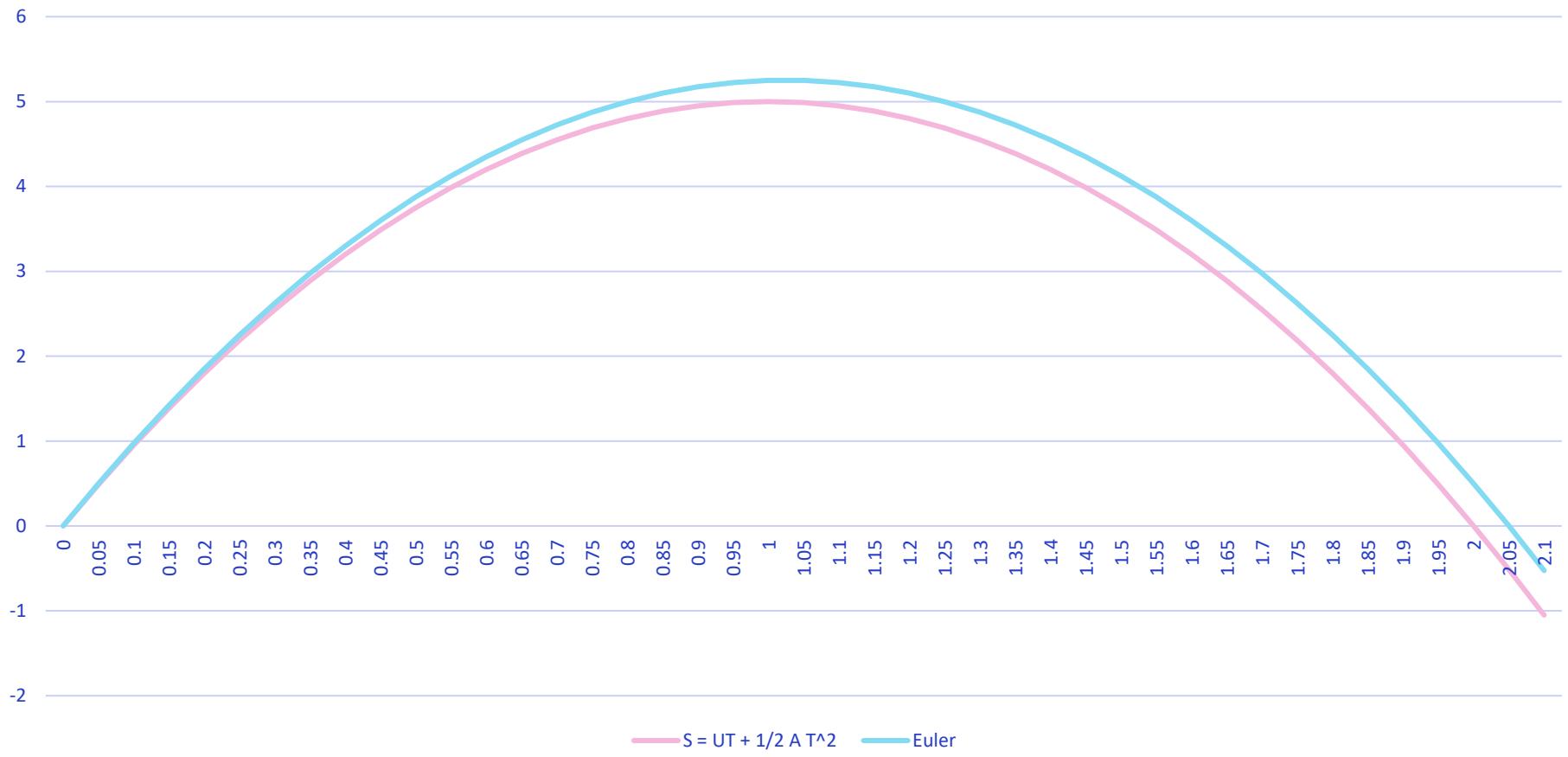
$k_3 = v_0 + (h/2) * k_2$

$k_4 = v_0 + h * k_3$



Time to “Code”

Canvas Workshop 2.1 Integration Tests



- See https://gafferongames.com/post/integration_basics/

Projectile/Particle Motion

Sir Isaac Newton

1. A body remains at rest, or in motion at a constant speed in a straight line, except insofar as it is acted upon by a force.
2. At any instant of time, the net force on a body is equal to the body's acceleration multiplied by its mass or, equivalently, the rate at which the body's momentum is changing with time.
3. If two bodies exert forces on each other, these forces have the same magnitude but opposite directions.

From a portrait by Kneller in 1702
This is in the National Portrait Gallery in London

Newton's Law

- 1st law - An object at rest stays at rest and an object in motion stays in motion with the same speed and in the same direction unless acted upon by an unbalanced force
- 2nd law- Force = mass x velocity / time
  - Force = mass x acceleration
- 3rd law - Every action has an equal and opposite reaction
- $F_g = G\ m_1\ m_2 / r^2$
  - Remember though we like to cut corners
  - Force = mass x acceleration where acceleration is 9.81 ms^{-2}

Sir Isaac Newton

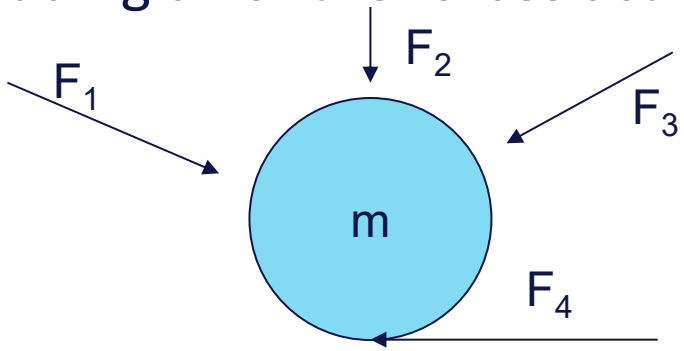
- Derived equations
  - new_velocity = old_velocity + acceleration*time
    - $v = u + at$
  - Distance_moved = old_velocity*time + ½*accel*time^2
    - $s = ut + \frac{1}{2}at^2$
  - A further equation is:
    - $v^2 = u^2 + 2as$
- Assumption
  - Constant acceleration a over the time period t

Simplifying the problem – Projectile/Particle Motion

- Linear motion
- No rotation
- Treat everything as a point mass
- Initially no forces
- Then add gravity
- Then sum forces

Adding Forces

- Gravity
  - $F_g = G\, m_1\, m_2 / r^2$
  - $F_g = m * 9.81\ ms^{-2}$
- Then other forces
- Acceleration (a) is found by adding all of the forces acting upon the object in question and dividing by its mass (m)



$$a = (F_1 + F_2 + F_3 + F_4)/m$$

In general $a = \Sigma F_i/m$

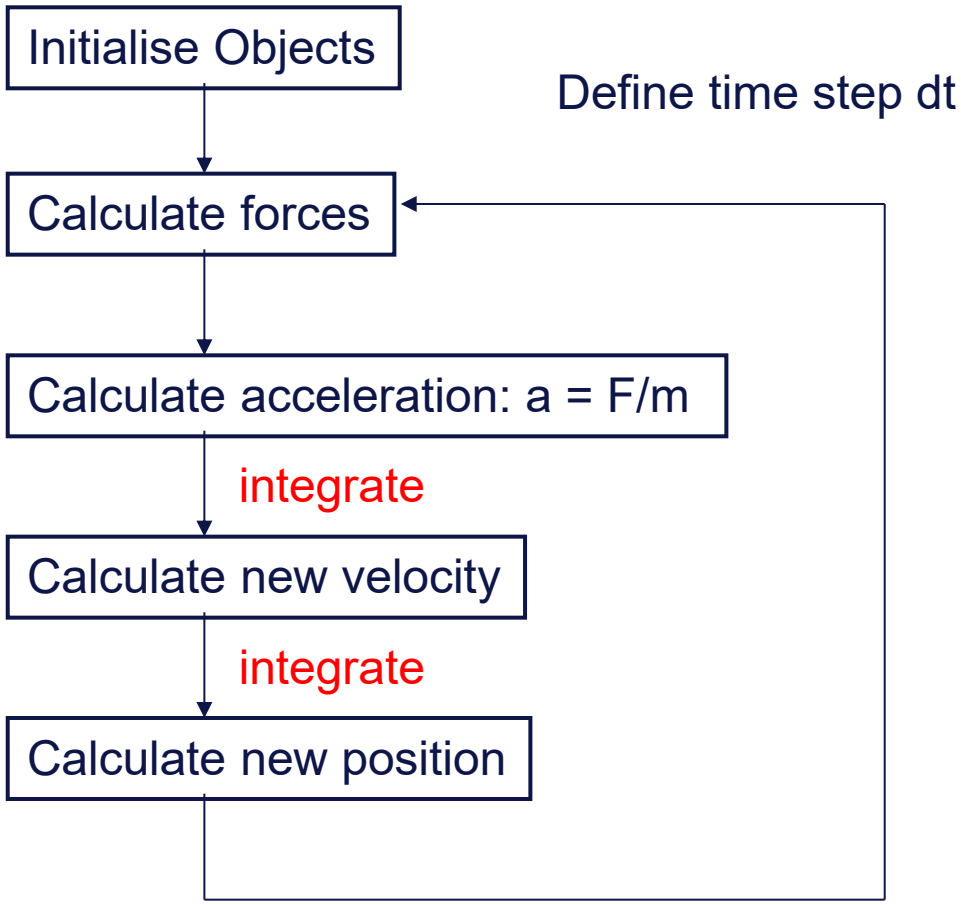
Laws of Motion

- Force = mass x acceleration
- Derived equations
  - new_velocity = old_velocity + acceleration*time
    - $v = u + at$
  - Distance_moved = old_velocity*time + ½*accel*time^2
    - $s = ut + \frac{1}{2}at^2$
  - A further equation is:
    - $v^2 = u^2 + 2as$
- Assumption
  - Constant acceleration a over the time period t

Laws of Motion

- Numerical Integration
  - Frequently we will NOT use these equations. Instead numerical techniques will be used to find velocity and distance moved directly from the forces acting upon a body
- Independent Study:
  - Numerical Recipes in C/C++
    - Read chapter 16 on “Integration of Ordinary Differential Equations”
  - Physics for Games Developers
    - Read chapter 1

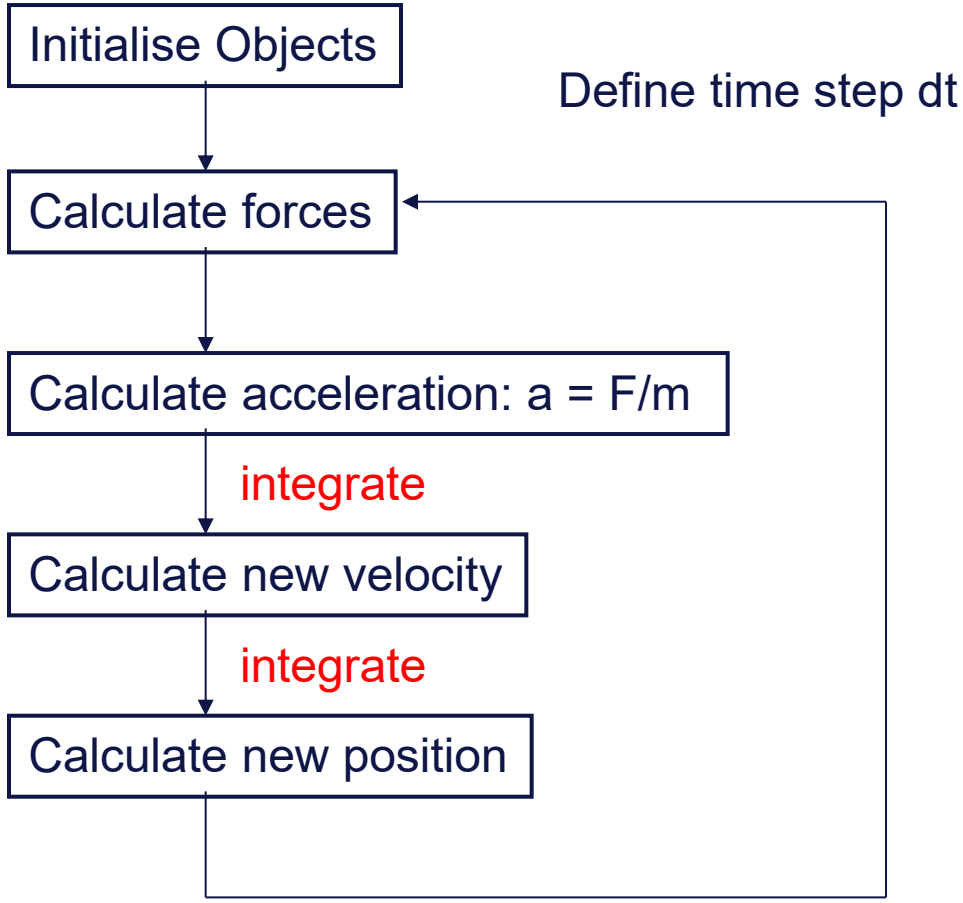
Simple Simulation Loop



Note: No collision detection/response included at the moment

Simple Simulation Loop

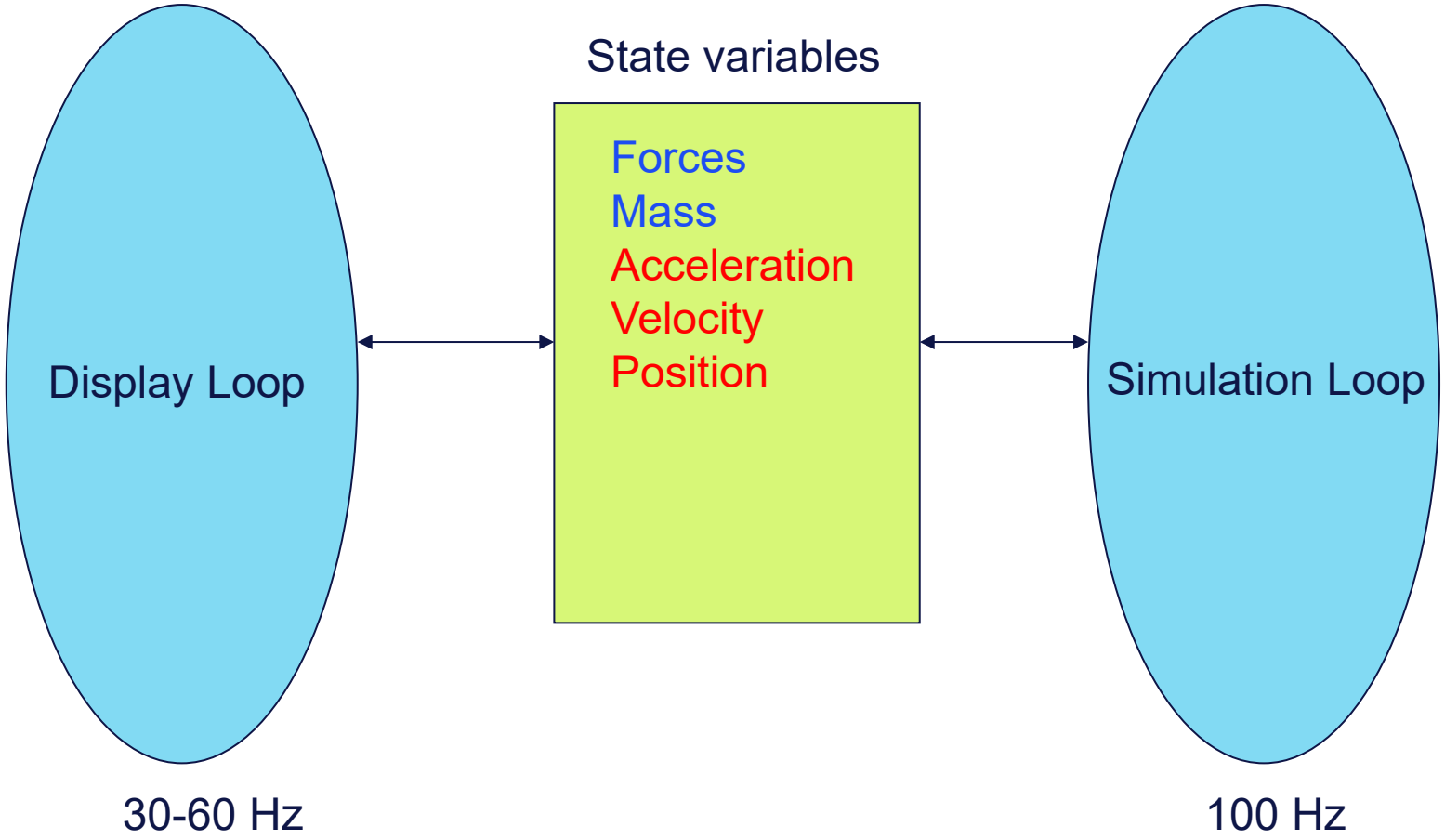
- dt means “delta time” which means a small change in time
- dt can be
  - fixed – a set value
  - variable – it can change depending on what else is going on



Fixed versus Variable Timestep

- Fixed time step:
  - When predictable behavior is crucial, like in ***game physics*** where consistent updates are needed for smooth gameplay.
  - When the system changes relatively uniformly and doesn't experience rapid fluctuations.
- Variable time step:
  - When high accuracy is essential, especially in simulations with dynamic changes where some parts of the system might change rapidly while others change slowly.
  - When optimizing simulation speed is important, as it can skip unnecessary calculations during stable periods.

Simulation & Display Loops



Rates are only for example

Time to Code

Canvas Workshop 2.2 Building the sandbox

Independent Study

- Gaffer on games does a bunch of really well explained articles
- https://gafferongames.com/post/integration_basics/
- https://gafferongames.com/post/fix_your_timestep/